Trees

Traversals, diameter, subtree sizes, tree DP and a first look at rerooting

Learning goals

- Root a tree and compute **parent, depth, subtree size** in one DFS
- Traverse without recursion using a **processing order**
- Find the **diameter** with two BFS/DFS passes, and prove it works
- Design **tree DP**: state per vertex, combine children
- Solve maximum matching / independent set on trees
- Use **rerooting** to get an answer for every root in O(n)

What is a tree?

Equivalent definitions for a graph on n vertices:

- connected and **n − 1 edges**
- connected and **acyclic**
- **exactly one** simple path between every pair of vertices

Consequences:

- No visited array needed: just don't go back to the **parent**
- Removing any edge splits it into exactly two parts
- Many problems that are hard on graphs are **easy on trees** (DP!)

Input formats

```
Edge list (n − 1 lines "a b")          Parent array "p₂ p₃ … pₙ"
5                                      5
1 2                                    1 1 2 3
1 3                                    (boss of 2 is 1, of 3 is 1,
3 4                                     of 4 is 2, of 5 is 3)
3 5
```

Edge list → undirected adjacency list, pick a root (usually 1).

Parent array → already rooted: `children[p].push_back(i)`.

Rooting the tree: DFS

```
void dfs(int u, int p) {
    par[u] = p;
    sz[u] = 1;
    for (int v : adj[u]) {
        if (v == p) continue;          // don't walk back up
        depth[v] = depth[u] + 1;
        dfs(v, u);
        sz[u] += sz[v];                // post-order: child is finished
    }
}
dfs(1, 0);
```

Pre-order work (before children): depth, parent.

Post-order work (after children): sizes, DP values.

Recursion depth on trees

A path ("bamboo") of $2 \cdot 10^5$ vertices → recursion depth $2 \cdot 10^5$.

- Codeforces / CSES: large stack → usually OK
- Other judges / local Windows: may crash

Iterative pattern (used in all reference code today):

1. Build `order` with an explicit stack; parents appear before children
2. Loop `order` **forward** → top-down computations (depth)
3. Loop `order` **backward** → bottom-up computations (sizes, DP)

Iterative order code

```
vector<int> order, par(n + 1, 0);
vector<int> st = {1};
par[1] = -1;
while (!st.empty()) {
    int u = st.back(); st.pop_back();
    order.push_back(u);
    for (int v : adj[u])
        if (v != par[u]) { par[v] = u; st.push_back(v); }
}
// bottom-up:
for (int i = n - 1; i > 0; i--) sz[par[order[i]]] += sz[order[i]];
```

Subtree sizes: Subordinates (CSES 1674)

Company tree rooted at 1; for each employee, count all subordinates (direct or indirect).

```
      1              sz: 1→5, 2→2, 3→2, 4→1, 5→1
     / \
    2   3            answer = sz − 1:
    |   |            4 1 1 0 0
    4   5
```

`sz[u] = 1 + Σ sz[child]` — the simplest tree DP.

Diameter

Diameter = longest path between any two vertices (in edges or total weight).

Two-pass algorithm:

1. From any vertex s, find the farthest vertex **a**
2. From a, find the farthest vertex **b**
3. dist(a, b) is the diameter

Two BFS (unweighted) or two DFS (weighted, non-negative) → **O(n)**.

Why the two-pass trick works

Claim: the farthest vertex a from any s is an endpoint of **some** diameter.

Sketch: let (u, v) be a diameter. If a is farther from s than both u and v, then splicing paths (s → a vs. the diameter path) produces a path longer than (u, v) — contradiction. Every case reduces to this exchange because paths in a tree are unique.

⚠ Fails with **negative** edge weights → use DP instead (next slides).

Bonus (CSES 1132): farthest distance from every v = max(dist(v, a), dist(v, b)).

Diameter by DP

Root the tree. `down[u]` = longest downward path from u.

```
down[u] = max over children c of (down[c] + w(u, c))     (0 for a leaf)
through u = sum of the two largest (down[c] + w) values
diameter = max over u of "through u"
```

One post-order pass, works with any weights, and is the **template** for tree DP.

Tree DP: the recipe

1. Root the tree
2. **State**: `dp[u][...]` describes the best/count **inside the subtree of u**, often with a flag about u itself
3. **Combine** children's values (they are independent subtrees!)
4. Evaluate in **post-order**
5. Answer at the root (or max over all u)

Children's subtrees are disjoint → no overlap issues, simple merges.

Example: maximum matching (CSES 1130)

Choose max number of edges, no two sharing a vertex.

- `dp0[u]` : best in subtree(u), **u not matched** to a child
- `dp1[u]` : best in subtree(u), **u matched** to exactly one child c

```
dp0[u] = Σ max(dp0[c], dp1[c])
dp1[u] = dp0[u] + max over c of ( dp0[c] + 1 − max(dp0[c], dp1[c]) )
```

Answer: max(dp0[root], dp1[root]). O(n).

Matching DP code

```
for (int i = n - 1; i >= 0; i--) {          // children before parents
    int u = order[i];
    int bestGain = INT_MIN;
    for (int c : adj[u]) {
        if (c == par[u]) continue;
        int bestC = max(dp0[c], dp1[c]);
        dp0[u] += bestC;
        bestGain = max(bestGain, dp0[c] + 1 - bestC);
    }
    if (bestGain != INT_MIN) dp1[u] = dp0[u] + bestGain;   // leaf: stays 0
}
cout << max(dp0[1], dp1[1]) << "\n";
```

Example: independent sets (AtCoder DP P)

Colour each vertex white/black, no two adjacent black. Count colourings mod 10^9+7.

```
white[u] = Π over children c of (white[c] + black[c])
black[u] = Π over children c of  white[c]
answer   = white[root] + black[root]
```

Same pattern: a flag for "what happens at u", product/sum/max over independent children.

Rerooting: an answer for every root

CSES 1133 Tree Distances II: for **every** v, sum of distances to all other vertices.

- Naive: n BFS → $O(n^2)$ ×
- Rerooting: compute the answer for root 1, then **move the root** across each edge in O(1)

Moving the root from p to its child c:

- the sz[c] vertices in c's subtree get **1 closer**
- the other n − sz[c] vertices get **1 farther**

```
ans[c] = ans[p] − sz[c] + (n − sz[c]) = ans[p] + n − 2·sz[c]
```

Rerooting code

```
// sz[] from a bottom-up pass, depth[] from top-down, both rooted at 1
for (int v = 1; v <= n; v++) ans[1] += depth[v];
for (int i = 1; i < n; i++) {                  // order: parents first
    int c = order[i];
    ans[c] = ans[par[c]] + n - 2 * sz[c];
}
```

Sample (edges 1-2, 1-3, 3-4, 3-5): `6 9 5 8 8`.

General rerooting (for max, not sum) needs prefix/suffix tricks — Week 22 / World Finals track.

Complexity

Task	Time	Memory
Root, parent, depth, subtree size	O(n)	O(n)
Diameter (two passes or DP)	O(n)	O(n)
Tree DP with O(1) per child	O(n)	O(n)
Rerooting with O(1) per edge	O(n)	O(n)

Every edge is looked at a constant number of times.

Common pitfalls

- Using a visited array with a wrong parent check → skips or revisits
- n = 1: no edges; diameter 0; don't read n − 1 = 0 lines wrongly
- Recursion depth on bamboo trees
- Weighted diameter with **negative** weights via two passes ×
- `long long` for sums of distances ($n^2 / 2 \approx 2 \cdot 10^{10}$)
- Forgetting the leaf case in DP (e.g. `dp1[leaf]` must not be $-\infty$ + something)
- Mixing up pre-order (top-down) and post-order (bottom-up) work

Practice problems

- CSES 1674 Subordinates · CSES 1131 Tree Diameter
- CSES 1132 Tree Distances I · CSES 1133 Tree Distances II
- CSES 1130 Tree Matching · AtCoder DP Contest P Independent Set
- CF 1092F Tree with Maximum Cost (rerooting)
- CF 580C Kefa and Park · CF 1528A Parsa's Humongous Tree
- AtCoder DP Contest V Subtree (rerooting, harder)

Summary

- Trees: unique paths, no cycles → track the parent, skip the visited array
- Pre-order for top-down info, post-order for bottom-up info; iterative `order` avoids stack issues
- Diameter: farthest from anywhere, then farthest from that
- Tree DP = per-vertex state (+ a flag) combining independent children
- Rerooting: shift the root across an edge with an O(1) update
- Next week: **DAGs**, topological sort, DP on DAGs and strongly connected components